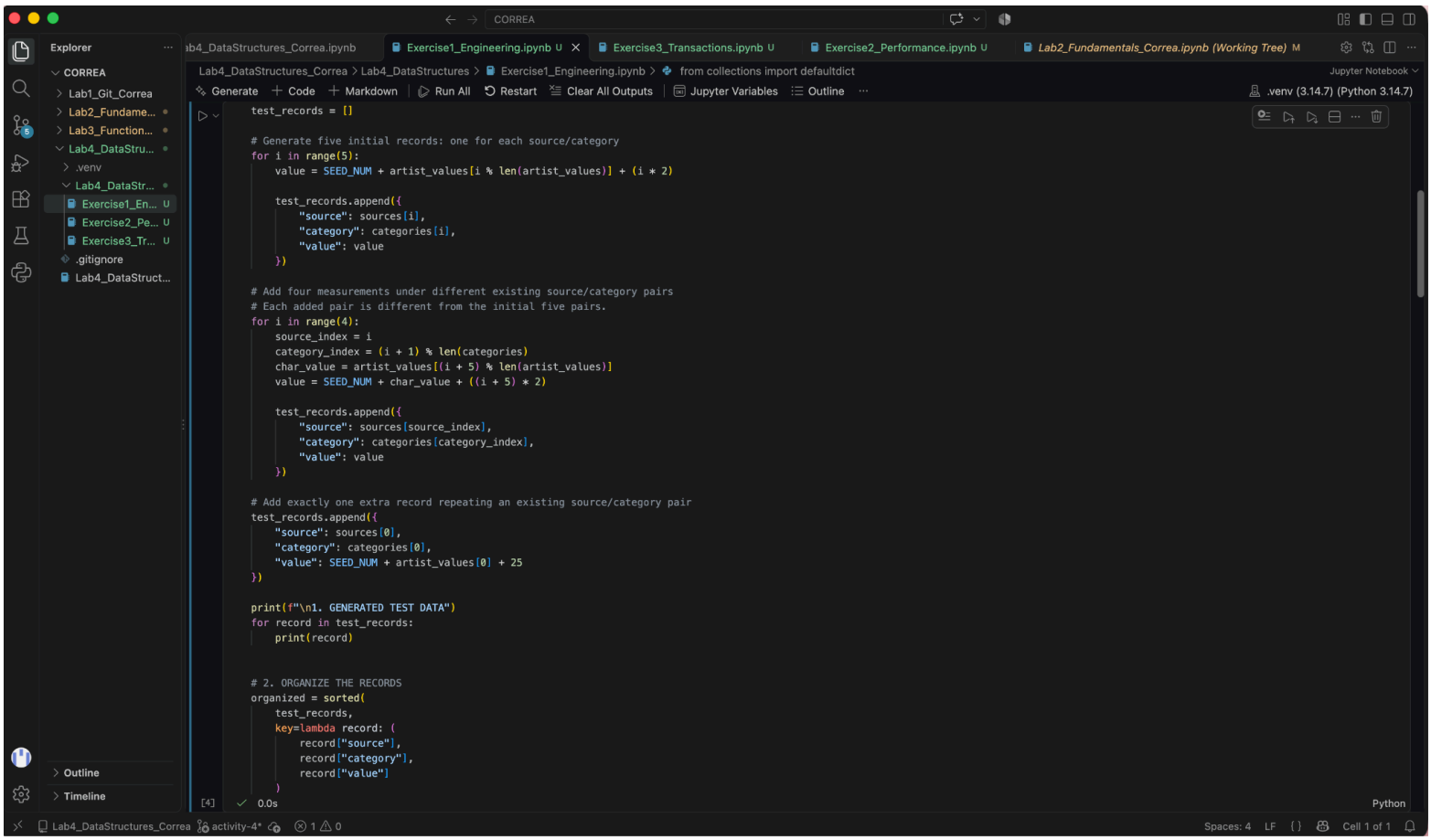
Princess Jherielle M. Correa
BSECE - 1C

October 11, 2026
Computer Programming

Activity 3

FUNCTIONS AND MODULES

EXERCISE #1 ENGINEERING TEST DATA ORGANIZER



```
test_records = []

# Generate five initial records: one for each source/category
for i in range(5):
    value = SEED_NUM + artist_values[i % len(artist_values)] + (i * 2)

    test_records.append({
        "source": sources[i],
        "category": categories[i],
        "value": value
    })

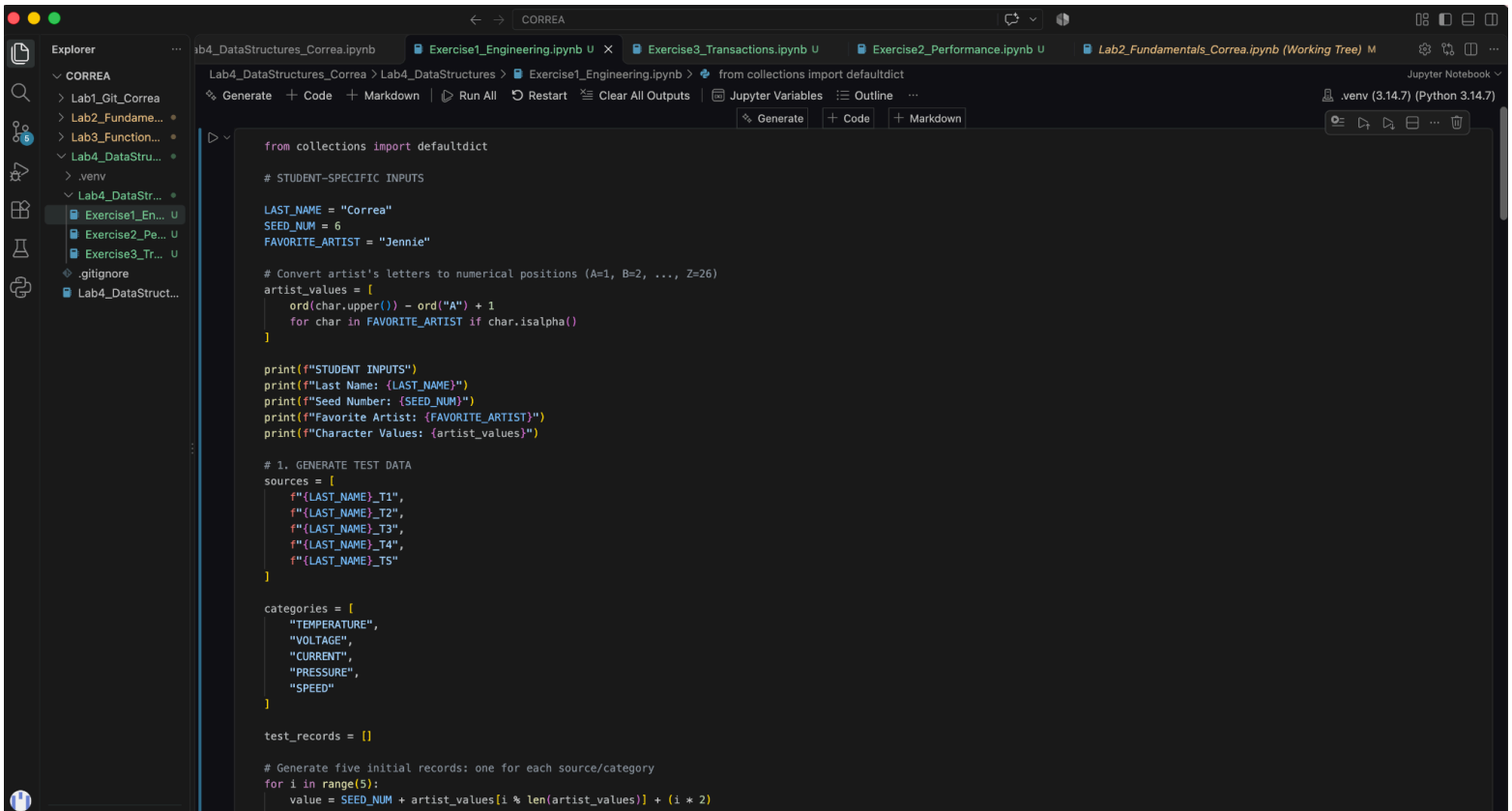
# Add four measurements under different existing source/category pairs
# Each added pair is different from the initial five pairs.
for i in range(4):
    source_index = i
    category_index = (i + 1) % len(categories)
    char_value = artist_values[(i + 5) % len(artist_values)]
    value = SEED_NUM + char_value + ((i + 5) * 2)

    test_records.append({
        "source": sources[source_index],
        "category": categories[category_index],
        "value": value
    })

# Add exactly one extra record repeating an existing source/category pair
test_records.append({
    "source": sources[0],
    "category": categories[0],
    "value": SEED_NUM + artist_values[0] + 25
})

print(f"\n1. GENERATED TEST DATA")
for record in test_records:
    print(record)


# 2. ORGANIZE THE RECORDS
organized = sorted(
    test_records,
    key=lambda record: (
        record["source"],
        record["category"],
        record["value"]
    )
```



```
from collections import defaultdict

# STUDENT-SPECIFIC INPUTS

LAST_NAME = "Correa"
SEED_NUM = 6
FAVORITE_ARTIST = "Jennie"

# Convert artist's letters to numerical positions (A=1, B=2, ..., Z=26)
artist_values = [
    ord(char.upper()) - ord("A") + 1
    for char in FAVORITE_ARTIST if char.isalpha()
]

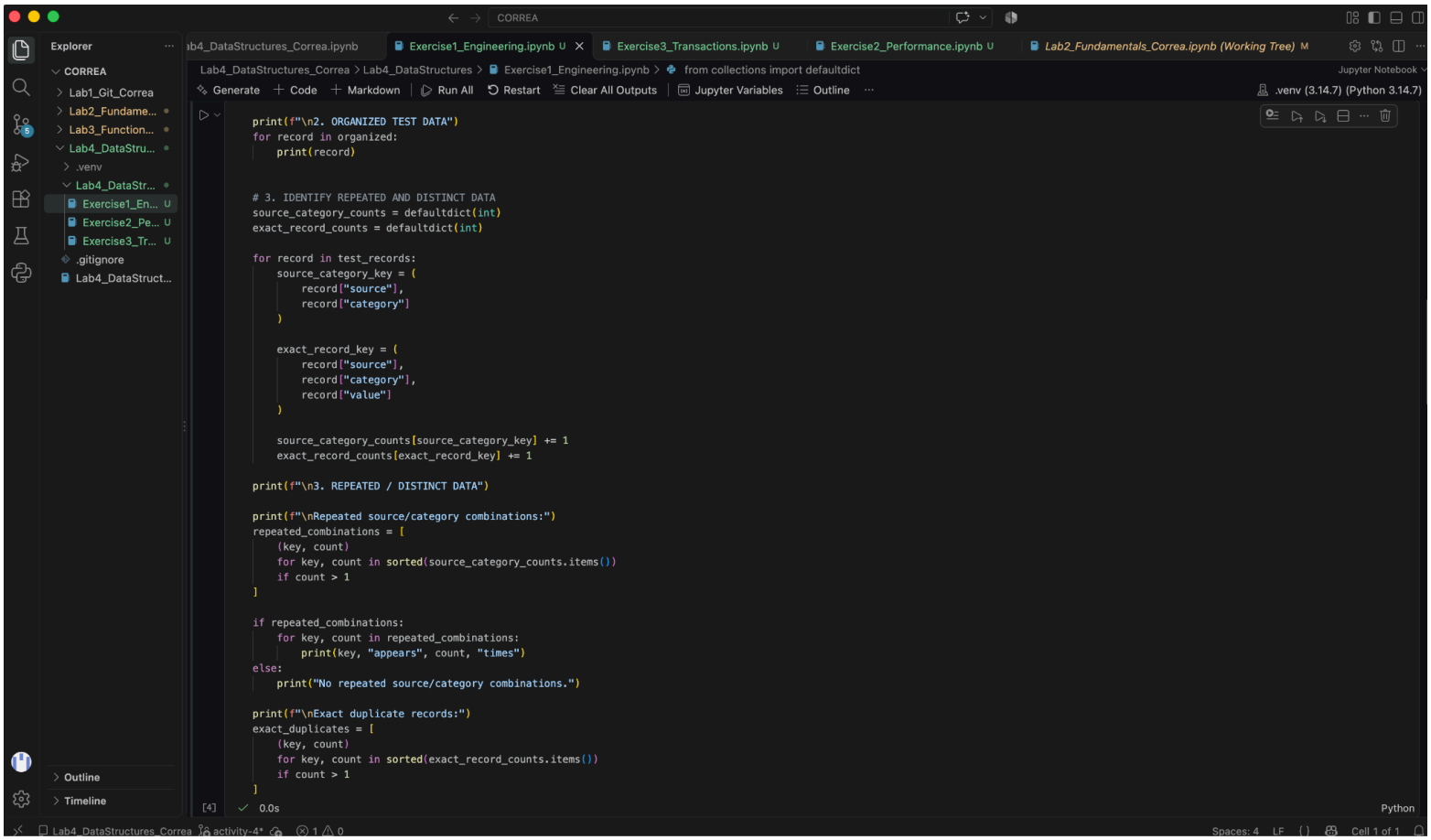
print(f"STUDENT INPUTS")
print(f"Last Name: {LAST_NAME}")
print(f"Seed Number: {SEED_NUM}")
print(f"Favorite Artist: {FAVORITE_ARTIST}")
print(f"Character Values: {artist_values}")

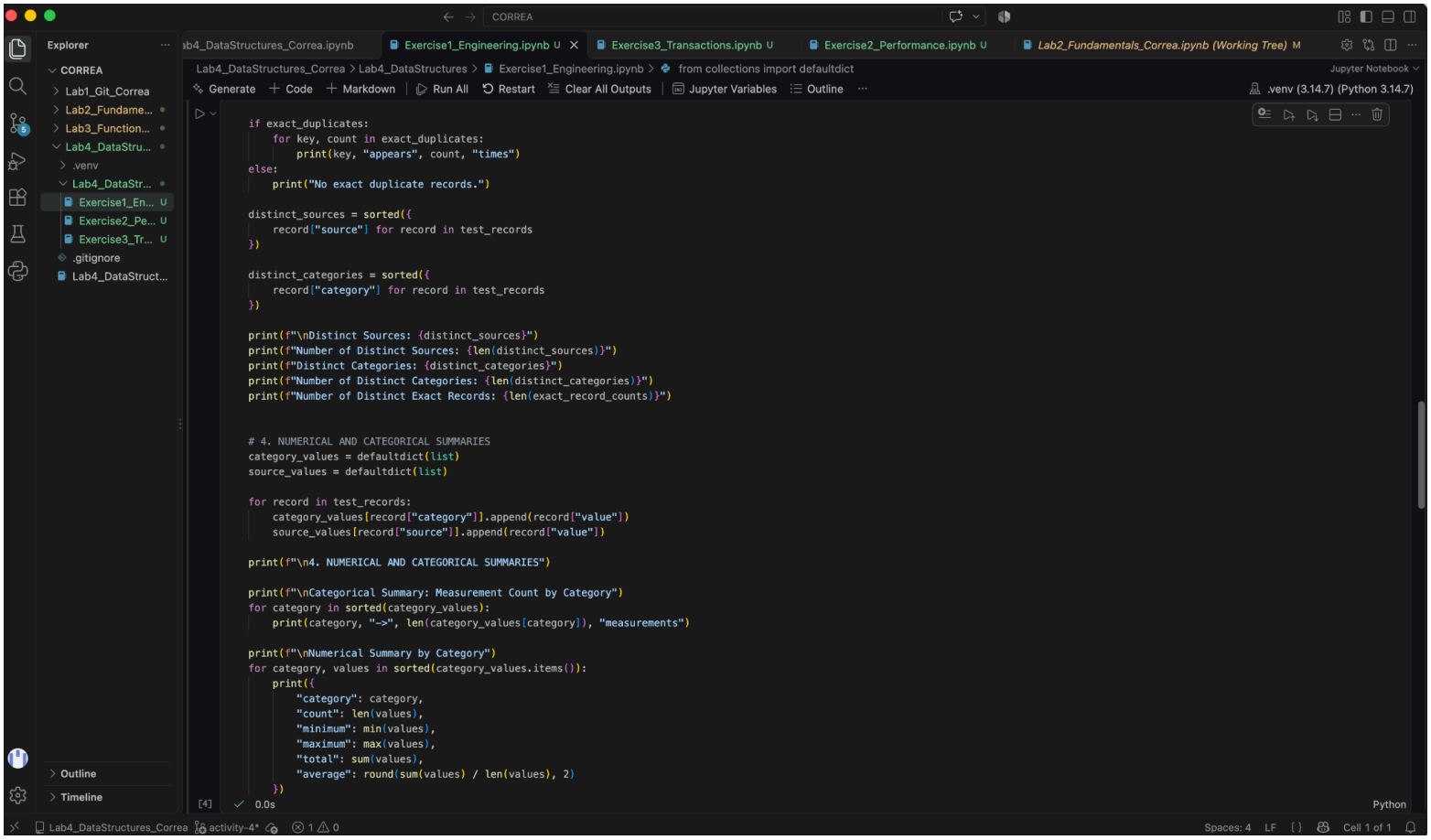
# 1. GENERATE TEST DATA
sources = [
    f"{LAST_NAME}_T1",
    f"{LAST_NAME}_T2",
    f"{LAST_NAME}_T3",
    f"{LAST_NAME}_T4",
    f"{LAST_NAME}_T5"
]

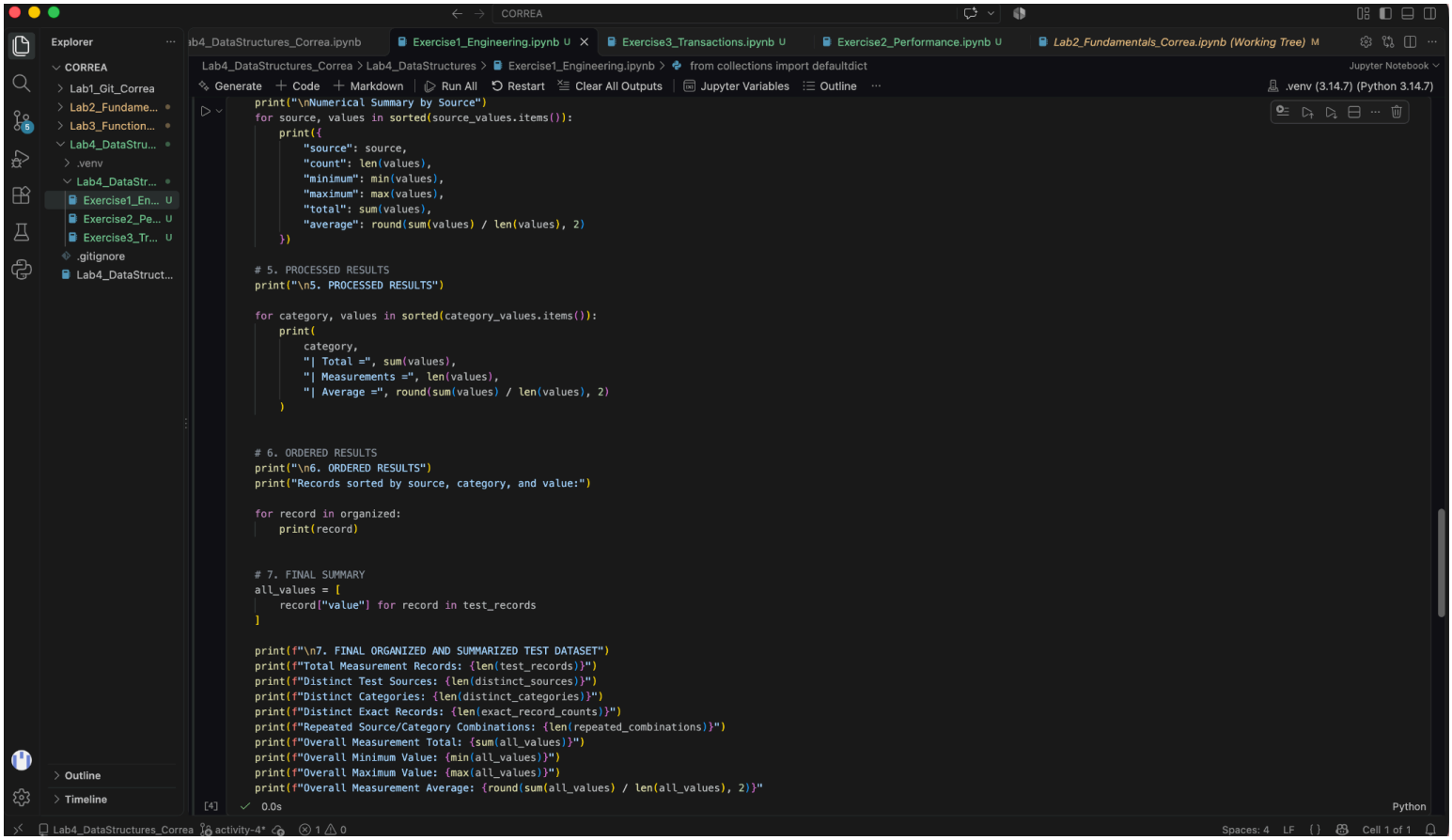
categories = [
    "TEMPERATURE",
    "VOLTAGE",
    "CURRENT",
    "PRESSURE",
    "SPEED"
]

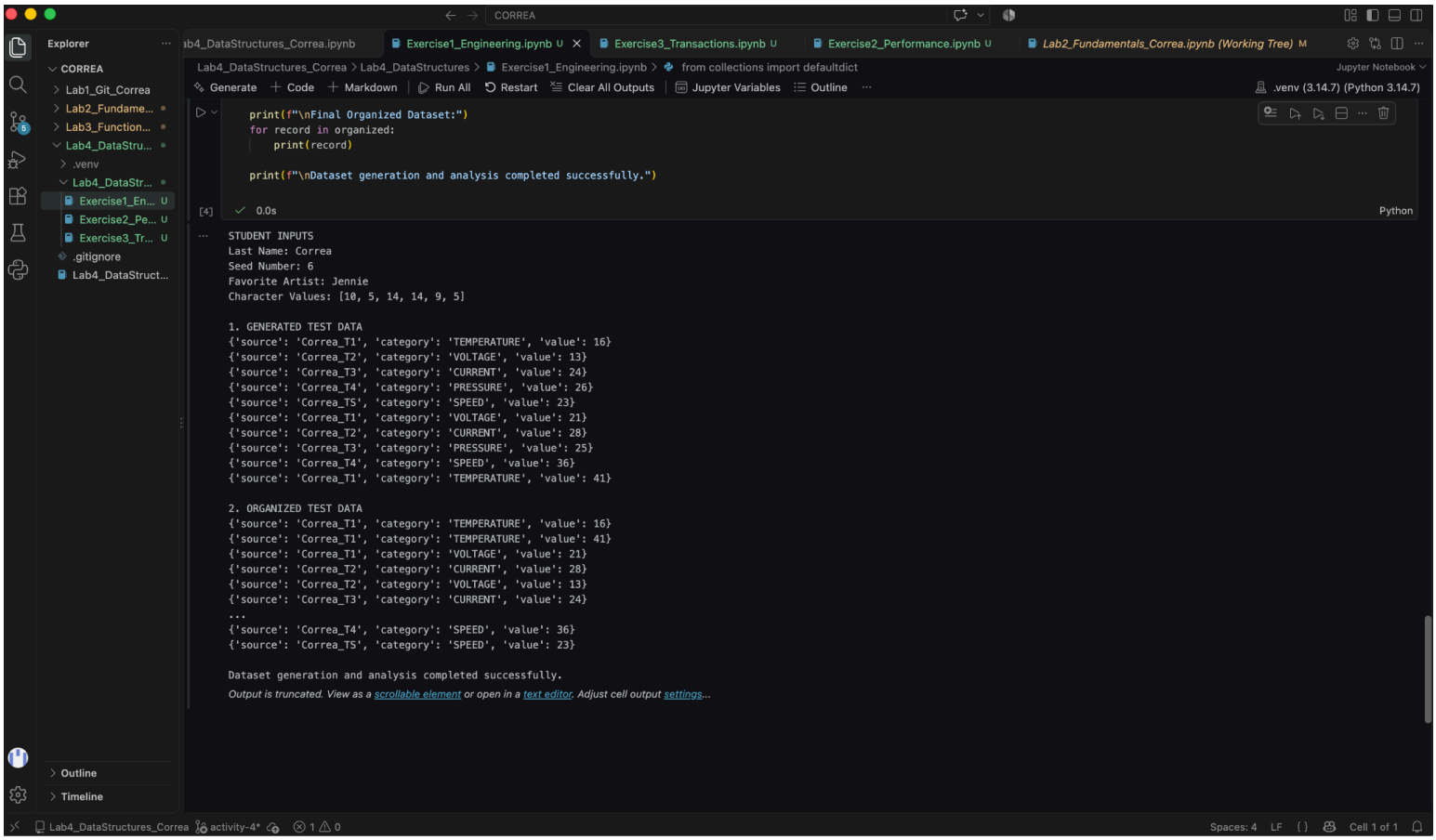
test_records = []

# Generate five initial records: one for each source/category
for i in range(5):
    value = SEED_NUM + artist_values[i % len(artist_values)] + (i * 2)
```

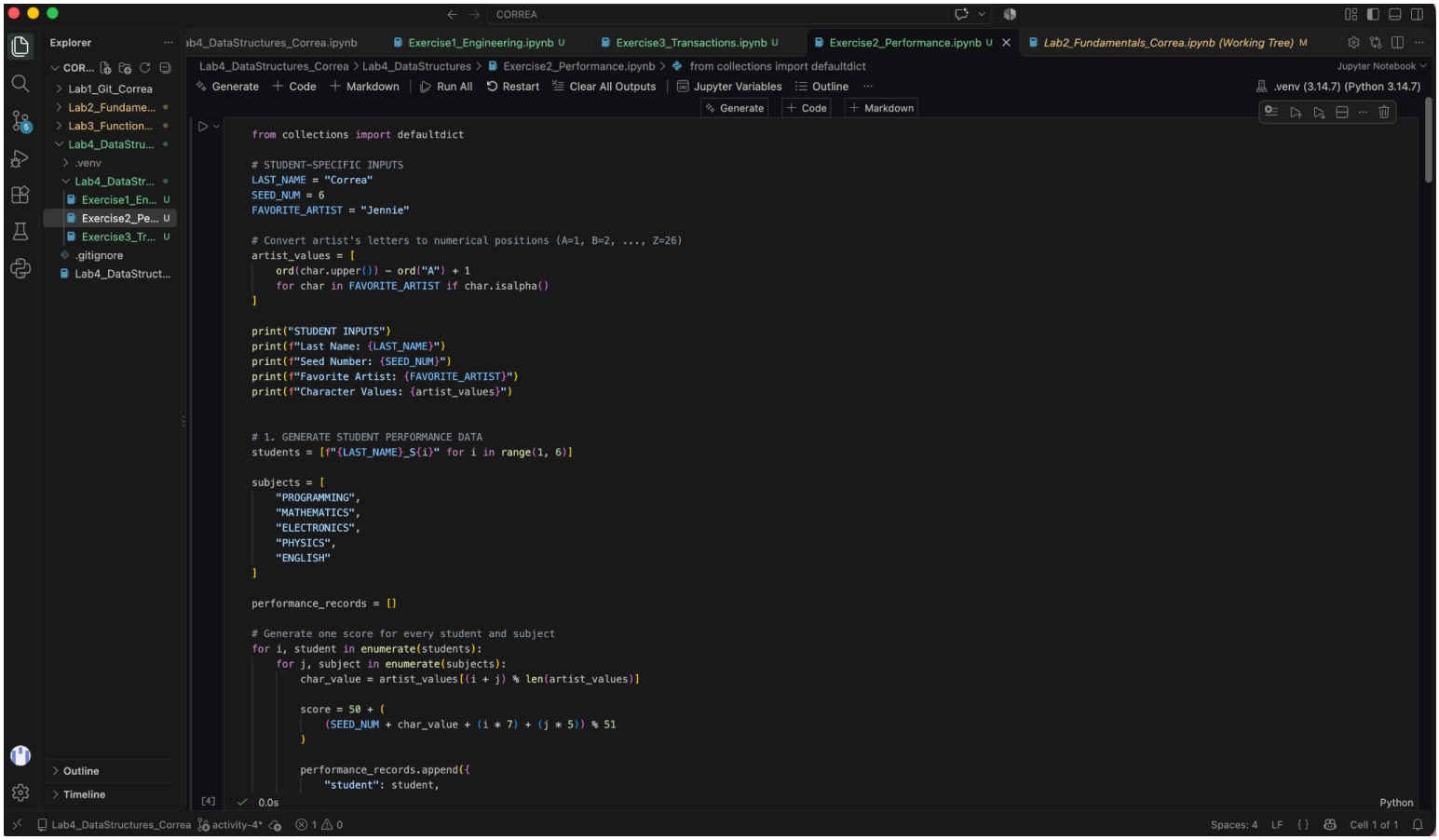

CORREA
Explorer
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb (Working Tree) M
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise1_Engineering.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
print(f"\n2. ORGANIZED TEST DATA")
for record in organized:
print(record)
3. IDENTIFY REPEATED AND DISTINCT DATA
source_category_counts = defaultdict(int)
exact_record_counts = defaultdict(int)
for record in test_records:
source_category_key = (
record["source"],
record["category"]
)
exact_record_key = (
record["source"],
record["category"],
record["value"]
)
source_category_counts[source_category_key] += 1
exact_record_counts[exact_record_key] += 1
print(f"\n3. REPEATED / DISTINCT DATA")
print(f"\nRepeated source/category combinations:")
repeated_combinations = [
(key, count)
for key, count in sorted(source_category_counts.items())
if count > 1
]
if repeated_combinations:
for key, count in repeated_combinations:
print(key, "appears", count, "times")
else:
print("No repeated source/category combinations.")
print(f"\nExact duplicate records:")
exact_duplicates = [
(key, count)
for key, count in sorted(exact_record_counts.items())
if count > 1
]
[4]
0.0s
Python
Outline
Timeline
Lab4_DataStructures_Correa activity-4*
Spaces: 4
LF
Cell 1 of 1


CORREA
Explorer
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb (Working Tree) M
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise1_Engineering.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
if exact_duplicates:
for key, count in exact_duplicates:
print(key, "appears", count, "times")
else:
print("No exact duplicate records.")
distinct_sources = sorted({
record["source"] for record in test_records
})
distinct_categories = sorted({
record["category"] for record in test_records
})
print(f"\nDistinct Sources: {distinct_sources}")
print(f"Number of Distinct Sources: {len(distinct_sources)}")
print(f"Distinct Categories: {distinct_categories}")
print(f"Number of Distinct Categories: {len(distinct_categories)}")
print(f"Number of Distinct Exact Records: {len(exact_record_counts)}")
4. NUMERICAL AND CATEGORICAL SUMMARIES
category_values = defaultdict(list)
source_values = defaultdict(list)
for record in test_records:
category_values[record["category"]].append(record["value"])
source_values[record["source"]].append(record["value"])
print(f"\n4. NUMERICAL AND CATEGORICAL SUMMARIES")
print(f"\nCategorical Summary: Measurement Count by Category")
for category in sorted(category_values):
print(category, "->", len(category_values[category]), "measurements")
print(f"\nNumerical Summary by Category")
for category, values in sorted(category_values.items()):
print({
"category": category,
"count": len(values),
"minimum": min(values),
"maximum": max(values),
"total": sum(values),
"average": round(sum(values) / len(values), 2)
})
[4]
0.0s
Python
Outline
Timeline
Lab4_DataStructures_Correa activity-4*
Spaces: 4
LF
Cell 1 of 1


CORREA
Explorer
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb (Working Tree) M
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise1_Engineering.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
print("\nNumerical Summary by Source")
for source, values in sorted(source_values.items()):
print({
"source": source,
"count": len(values),
"minimum": min(values),
"maximum": max(values),
"total": sum(values),
"average": round(sum(values) / len(values), 2)
})
5. PROCESSED RESULTS
print("\n5. PROCESSED RESULTS")
for category, values in sorted(category_values.items()):
print(
category,
"| Total =", sum(values),
"| Measurements =", len(values),
"| Average =", round(sum(values) / len(values), 2)
)
6. ORDERED RESULTS
print("\n6. ORDERED RESULTS")
print("Records sorted by source, category, and value:")
for record in organized:
print(record)
7. FINAL SUMMARY
all_values = [
record["value"] for record in test_records
]
print(f"\n7. FINAL ORGANIZED AND SUMMARIZED TEST DATASET")
print(f"Total Measurement Records: {len(test_records)}")
print(f"Distinct Test Sources: {len(distinct_sources)}")
print(f"Distinct Categories: {len(distinct_categories)}")
print(f"Distinct Exact Records: {len(exact_record_counts)}")
print(f"Repeated Source/Category Combinations: {len(repeated_combinations)}")
print(f"Overall Measurement Total: {sum(all_values)}")
print(f"Overall Minimum Value: {min(all_values)}")
print(f"Overall Maximum Value: {max(all_values)}")
print(f"Overall Measurement Average: {round(sum(all_values) / len(all_values), 2)}")
[4]
0.0s
Python
Outline
Timeline
Lab4_DataStructures_Correa
activity-4*
Spaces: 4
LF
Cell 1 of 1


CORREA
Explorer
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb (Working Tree) M
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise1_Engineering.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
print(f"\nFinal Organized Dataset:")
for record in organized:
print(record)
print(f"\nDataset generation and analysis completed successfully.")
[4]
0.0s
Python
STUDENT INPUTS
Last Name: Correa
Seed Number: 6
Favorite Artist: Jennie
Character Values: [10, 5, 14, 14, 9, 5]
1. GENERATED TEST DATA
{'source': 'Correa_T1', 'category': 'TEMPERATURE', 'value': 16}
{'source': 'Correa_T2', 'category': 'VOLTAGE', 'value': 13}
{'source': 'Correa_T3', 'category': 'CURRENT', 'value': 24}
{'source': 'Correa_T4', 'category': 'PRESSURE', 'value': 26}
{'source': 'Correa_T5', 'category': 'SPEED', 'value': 23}
{'source': 'Correa_T1', 'category': 'VOLTAGE', 'value': 21}
{'source': 'Correa_T2', 'category': 'CURRENT', 'value': 28}
{'source': 'Correa_T3', 'category': 'PRESSURE', 'value': 25}
{'source': 'Correa_T4', 'category': 'SPEED', 'value': 36}
{'source': 'Correa_T1', 'category': 'TEMPERATURE', 'value': 41}
2. ORGANIZED TEST DATA
{'source': 'Correa_T1', 'category': 'TEMPERATURE', 'value': 16}
{'source': 'Correa_T1', 'category': 'TEMPERATURE', 'value': 41}
{'source': 'Correa_T1', 'category': 'VOLTAGE', 'value': 21}
{'source': 'Correa_T2', 'category': 'CURRENT', 'value': 28}
{'source': 'Correa_T2', 'category': 'VOLTAGE', 'value': 13}
{'source': 'Correa_T3', 'category': 'CURRENT', 'value': 24}
...
{'source': 'Correa_T4', 'category': 'SPEED', 'value': 36}
{'source': 'Correa_T5', 'category': 'SPEED', 'value': 23}
Dataset generation and analysis completed successfully.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
Outline
Timeline
Lab4_DataStructures_Correa
activity-4*
Spaces: 4
LF
Cell 1 of 1

EXERCISE #2 STUDENT PERFORMANCE DATA ANALYZER



```python
from collections import defaultdict

# STUDENT-SPECIFIC INPUTS
LAST_NAME = "Correa"
SEED_NUM = 6
FAVORITE_ARTIST = "Jennie"

# Convert artist's letters to numerical positions (A=1, B=2, ..., Z=26)
artist_values = [
    ord(char.upper()) - ord("A") + 1
    for char in FAVORITE_ARTIST if char.isalpha()
]

print("STUDENT INPUTS")
print(f"Last Name: {LAST_NAME}")
print(f"Seed Number: {SEED_NUM}")
print(f"Favorite Artist: {FAVORITE_ARTIST}")
print(f"Character Values: {artist_values}")


# 1. GENERATE STUDENT PERFORMANCE DATA
students = [f"{LAST_NAME}_S{i}" for i in range(1, 6)]

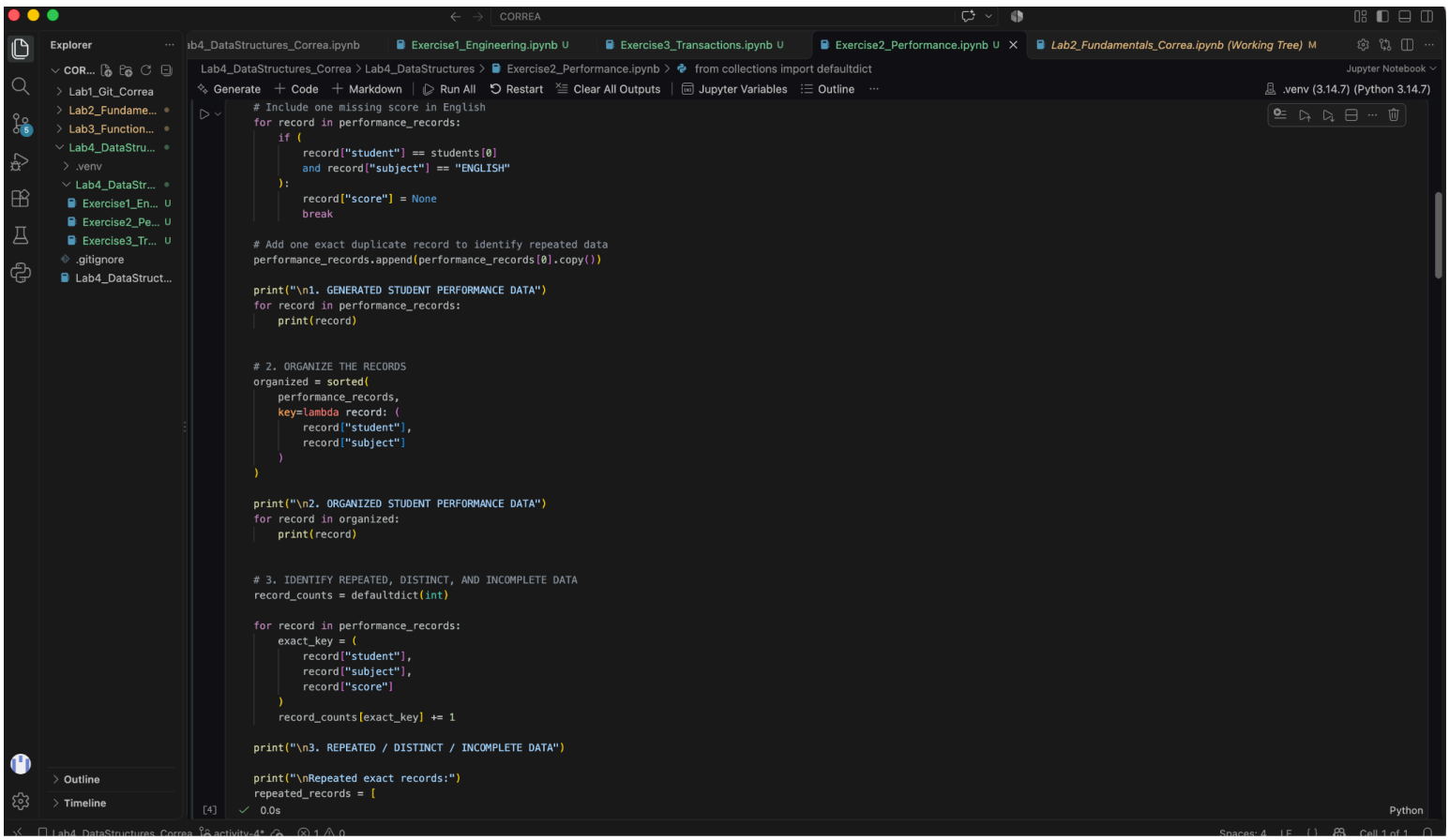
subjects = [
    "PROGRAMMING",
    "MATHEMATICS",
    "ELECTRONICS",
    "PHYSICS",
    "ENGLISH"
]

performance_records = []

# Generate one score for every student and subject
for i, student in enumerate(students):
    for j, subject in enumerate(subjects):
        char_value = artist_values[(i + j) % len(artist_values)]

        score = 50 + (
            (SEED_NUM + char_value + (i * 7) + (j * 5)) % 51
        )

        performance_records.append({
            "student": student,
```



```python
# Include one missing score in English
for record in performance_records:
    if (
        record["student"] == students[0]
        and record["subject"] == "ENGLISH"
    ):
        record["score"] = None
        break

# Add one exact duplicate record to identify repeated data
performance_records.append(performance_records[0].copy())

print("\n1. GENERATED STUDENT PERFORMANCE DATA")
for record in performance_records:
    print(record)


# 2. ORGANIZE THE RECORDS
organized = sorted(
    performance_records,
    key=lambda record: (
        record["student"],
        record["subject"]
    )
)

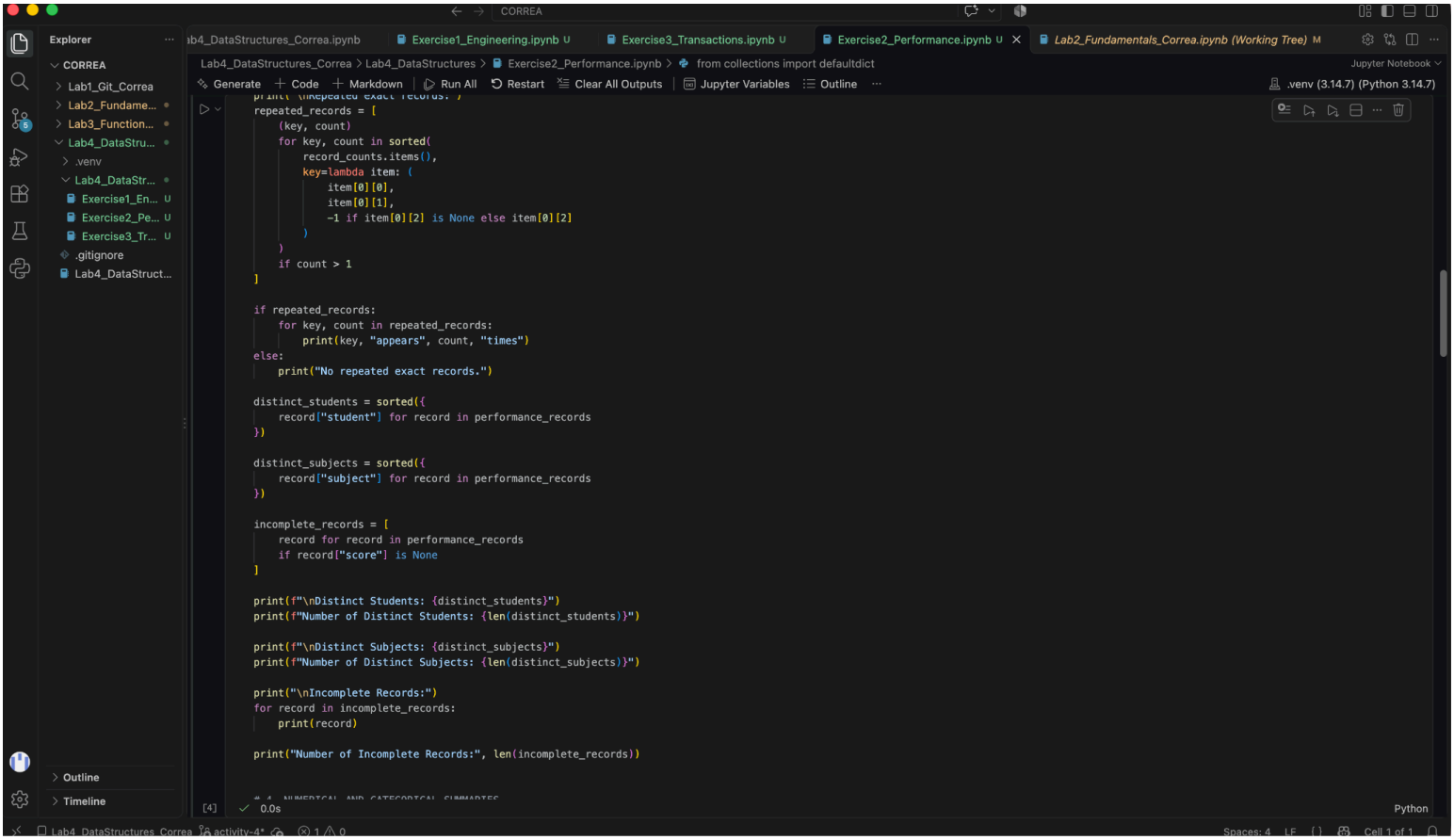
print("\n2. ORGANIZED STUDENT PERFORMANCE DATA")
for record in organized:
    print(record)

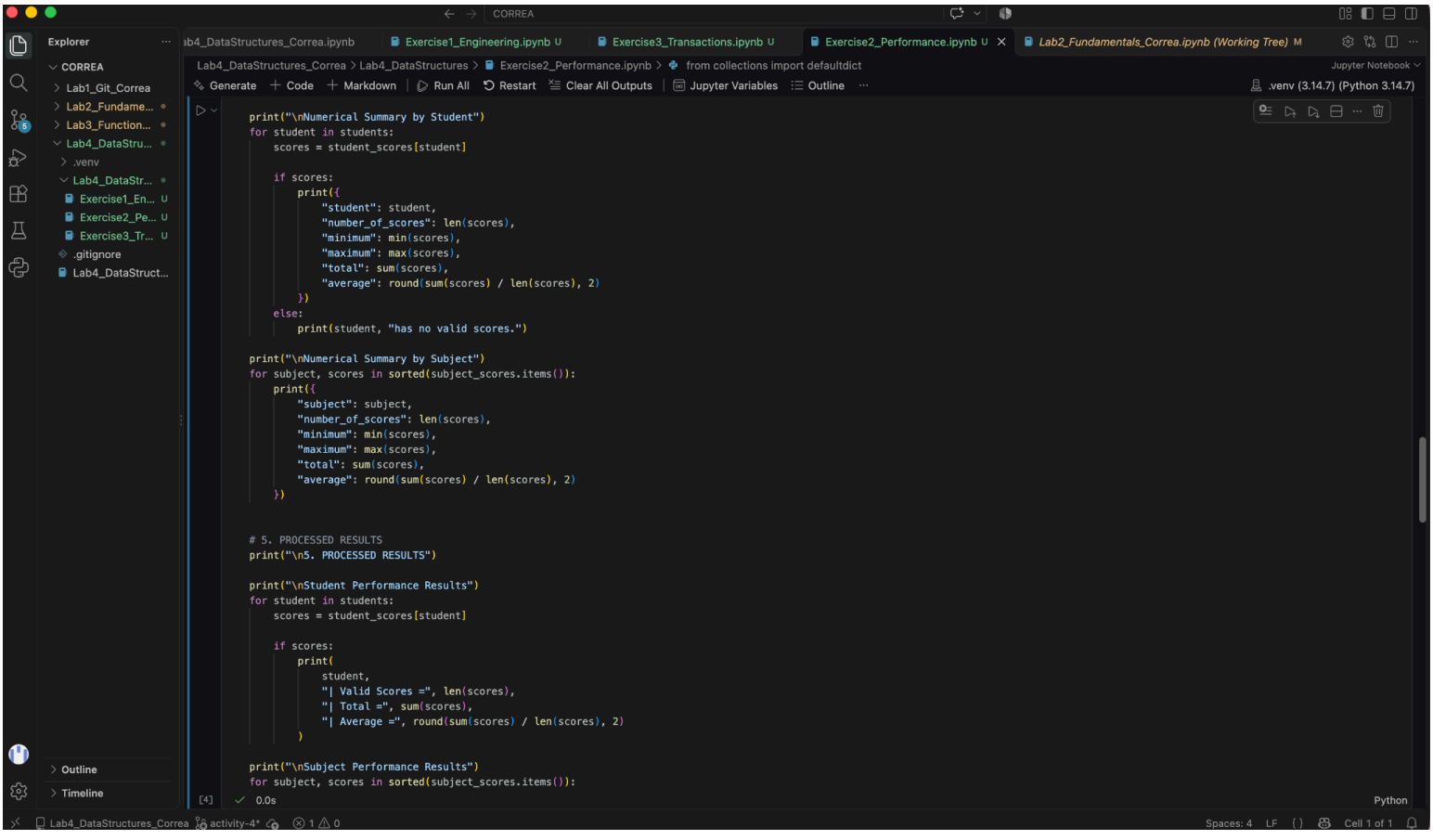

# 3. IDENTIFY REPEATED, DISTINCT, AND INCOMPLETE DATA
record_counts = defaultdict(int)

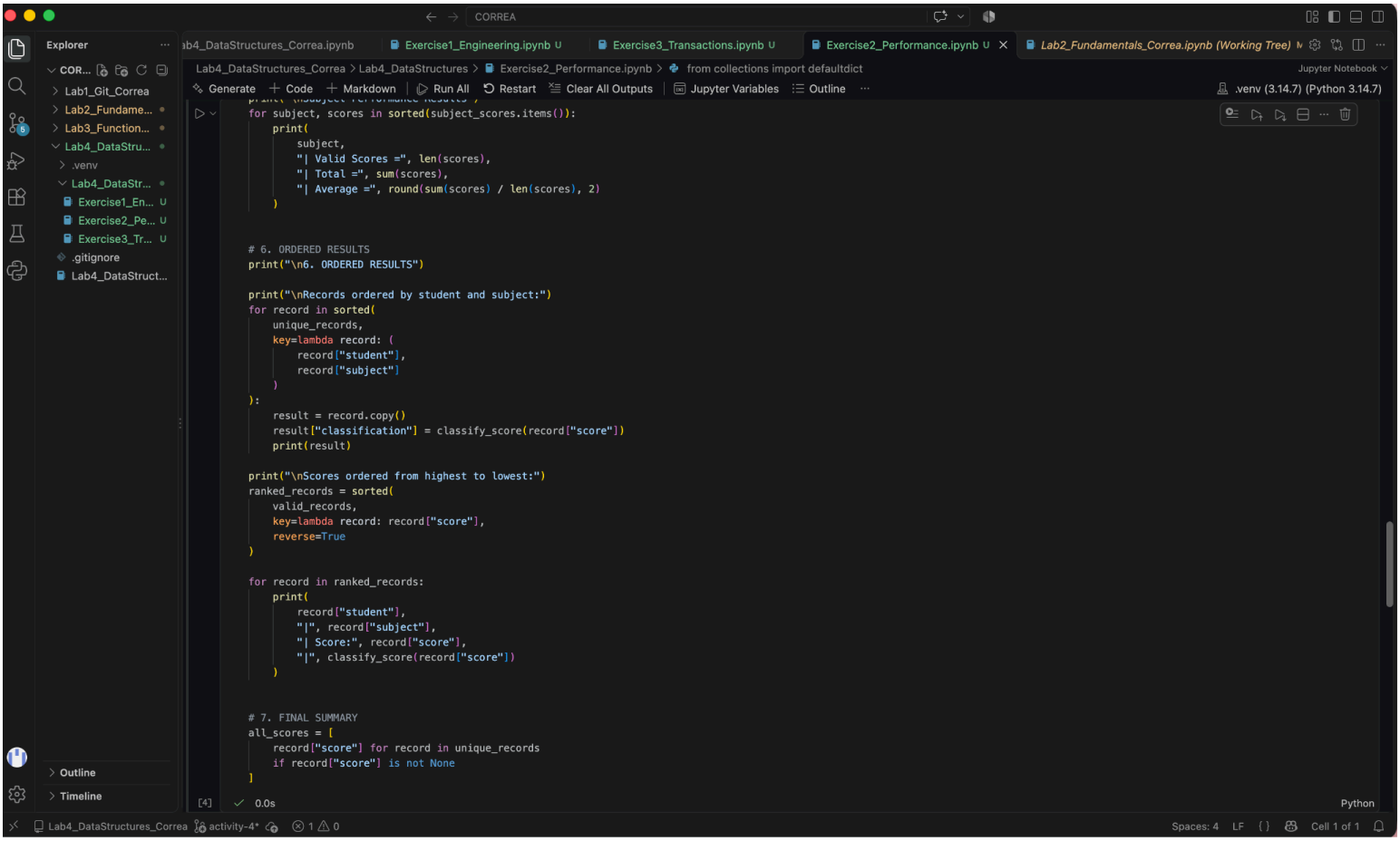
for record in performance_records:
    exact_key = (
        record["student"],
        record["subject"],
        record["score"]
    )
    record_counts[exact_key] += 1

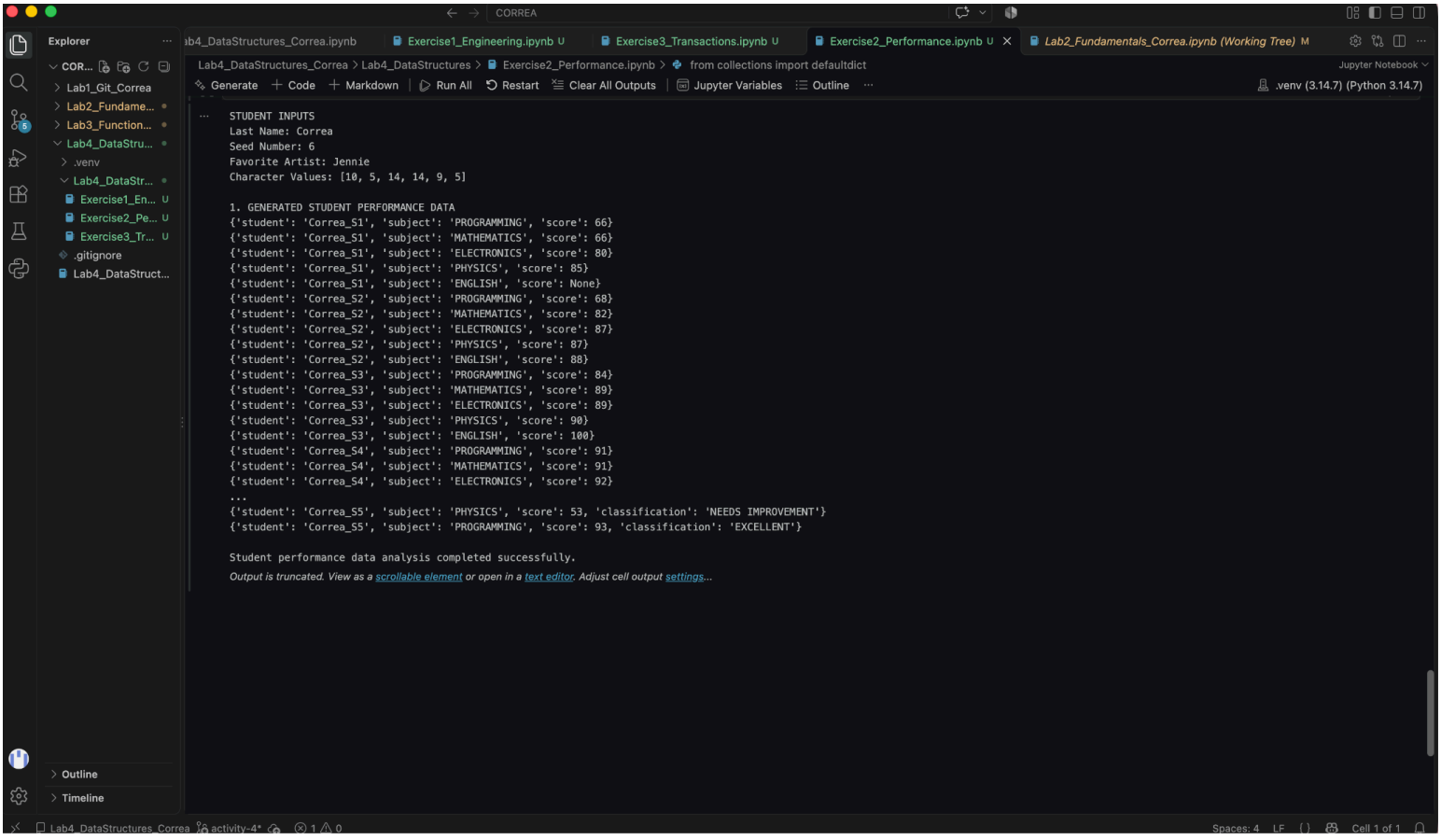
print("\n3. REPEATED / DISTINCT / INCOMPLETE DATA")

print("\nRepeated exact records:")
repeated_records = [
```

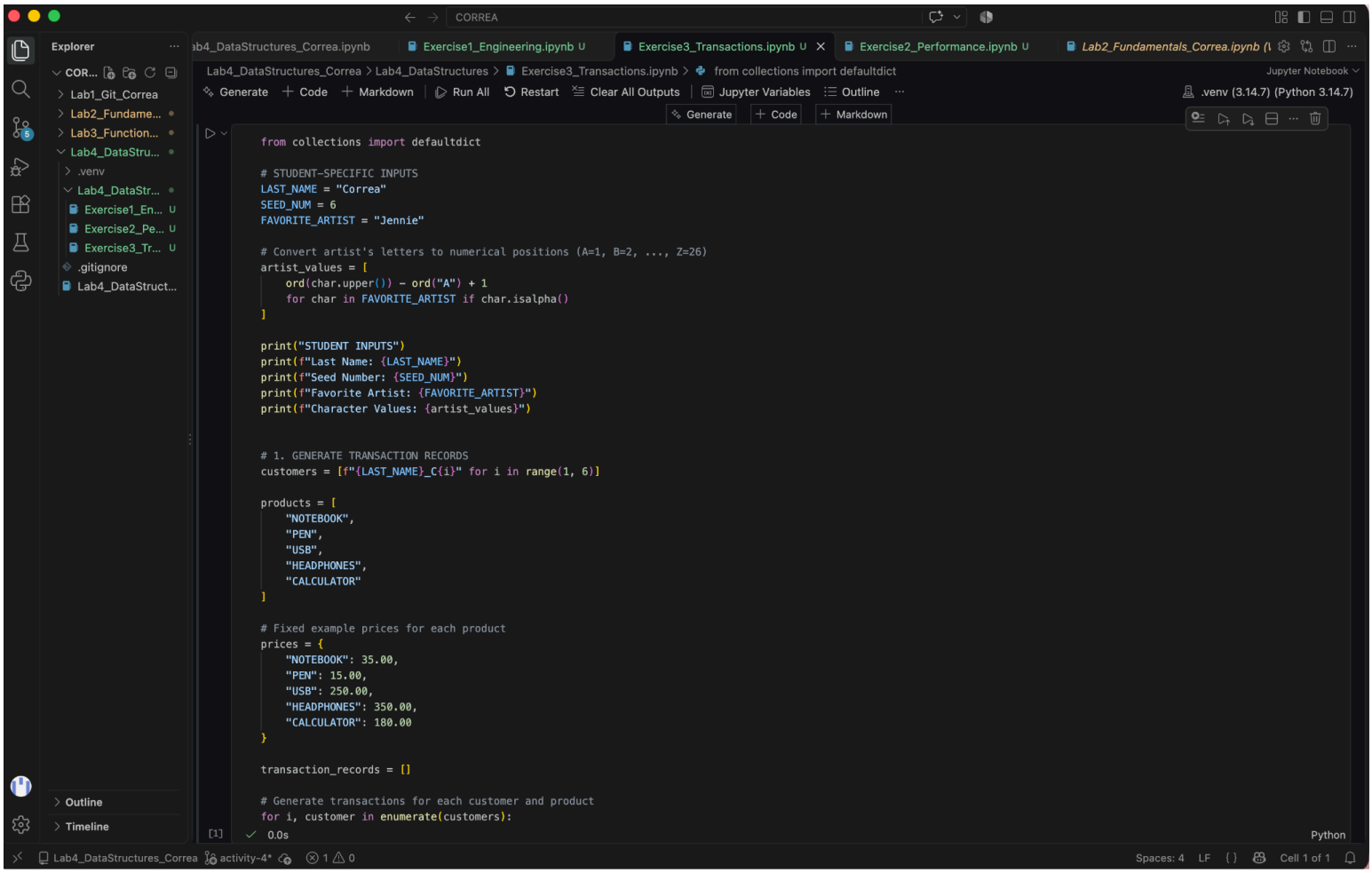

Explorer
CORREA
Lab1_Git_Correa
Lab2_Fundame...
Lab3_Function...
Lab4_DataStru...
.venv
Lab4_DataStr...
Exercise1_En... U
Exercise2_Pe... U
Exercise3_Tr... U
.gitignore
Lab4_DataStruct...
Outline
Timeline
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb (Working Tree) M
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise2_Performance.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
print("\nRepeated exact records: ")
repeated_records = [
(key, count)
for key, count in sorted(
record_counts.items(),
key=lambda item: (
item[0][0],
item[0][1],
-1 if item[0][2] is None else item[0][2]
)
)
if count > 1
]
if repeated_records:
for key, count in repeated_records:
print(key, "appears", count, "times")
else:
print("No repeated exact records.")
distinct_students = sorted({
record["student"] for record in performance_records
})
distinct_subjects = sorted({
record["subject"] for record in performance_records
})
incomplete_records = [
record for record in performance_records
if record["score"] is None
]
print(f"\nDistinct Students: {distinct_students}")
print(f"Number of Distinct Students: {len(distinct_students)}")
print(f"\nDistinct Subjects: {distinct_subjects}")
print(f"Number of Distinct Subjects: {len(distinct_subjects)}")
print("\nIncomplete Records:")
for record in incomplete_records:
print(record)
print("Number of Incomplete Records:", len(incomplete_records))
[4]
0.0s
Python
Lab4_DataStructures_Correa activity-4*
Spaces: 4
LF
Cell 1 of 1


Explorer
CORREA
Lab1_Git_Correa
Lab2_Fundame...
Lab3_Function...
Lab4_DataStru...
.venv
Lab4_DataStr...
Exercise1_En... U
Exercise2_Pe... U
Exercise3_Tr... U
.gitignore
Lab4_DataStruct...
Outline
Timeline
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb (Working Tree) M
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise2_Performance.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
print("\nNumerical Summary by Student")
for student in students:
scores = student_scores[student]
if scores:
print({
"student": student,
"number_of_scores": len(scores),
"minimum": min(scores),
"maximum": max(scores),
"total": sum(scores),
"average": round(sum(scores) / len(scores), 2)
})
else:
print(student, "has no valid scores.")
print("\nNumerical Summary by Subject")
for subject, scores in sorted(subject_scores.items()):
print({
"subject": subject,
"number_of_scores": len(scores),
"minimum": min(scores),
"maximum": max(scores),
"total": sum(scores),
"average": round(sum(scores) / len(scores), 2)
})
5. PROCESSED RESULTS
print("\n5. PROCESSED RESULTS")
print("\nStudent Performance Results")
for student in students:
scores = student_scores[student]
if scores:
print(
student,
"| Valid Scores =", len(scores),
"| Total =", sum(scores),
"| Average =", round(sum(scores) / len(scores), 2)
)
print("\nSubject Performance Results")
for subject, scores in sorted(subject_scores.items()):
[4]
0.0s
Python
Lab4_DataStructures_Correa activity-4*
Spaces: 4
LF
Cell 1 of 1


CORREA
Explorer
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb (Working Tree) M
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise2_Performance.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
for subject, scores in sorted(subject_scores.items()):
print(
subject,
"| Valid Scores =", len(scores),
"| Total =", sum(scores),
"| Average =", round(sum(scores) / len(scores), 2)
)
6. ORDERED RESULTS
print("\n6. ORDERED RESULTS")
print("\nRecords ordered by student and subject:")
for record in sorted(
unique_records,
key=lambda record: (
record["student"],
record["subject"]
)
):
result = record.copy()
result["classification"] = classify_score(record["score"])
print(result)
print("\nScores ordered from highest to lowest:")
ranked_records = sorted(
valid_records,
key=lambda record: record["score"],
reverse=True
)
for record in ranked_records:
print(
record["student"],
"|", record["subject"],
"| Score:", record["score"],
"|", classify_score(record["score"])
)
7. FINAL SUMMARY
all_scores = [
record["score"] for record in unique_records
if record["score"] is not None
]
[4]
0.0s
Python
Outline
Timeline
Lab4_DataStructures_Correa
activity-4*
Spaces: 4
LF
Cell 1 of 1


CORREA
Explorer
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb (Working Tree) M
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise2_Performance.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
STUDENT INPUTS
Last Name: Correa
Seed Number: 6
Favorite Artist: Jennie
Character Values: [10, 5, 14, 14, 9, 5]
1. GENERATED STUDENT PERFORMANCE DATA
{'student': 'Correa_S1', 'subject': 'PROGRAMMING', 'score': 66}
{'student': 'Correa_S1', 'subject': 'MATHEMATICS', 'score': 66}
{'student': 'Correa_S1', 'subject': 'ELECTRONICS', 'score': 80}
{'student': 'Correa_S1', 'subject': 'PHYSICS', 'score': 85}
{'student': 'Correa_S1', 'subject': 'ENGLISH', 'score': None}
{'student': 'Correa_S2', 'subject': 'PROGRAMMING', 'score': 68}
{'student': 'Correa_S2', 'subject': 'MATHEMATICS', 'score': 82}
{'student': 'Correa_S2', 'subject': 'ELECTRONICS', 'score': 87}
{'student': 'Correa_S2', 'subject': 'PHYSICS', 'score': 87}
{'student': 'Correa_S2', 'subject': 'ENGLISH', 'score': 88}
{'student': 'Correa_S3', 'subject': 'PROGRAMMING', 'score': 84}
{'student': 'Correa_S3', 'subject': 'MATHEMATICS', 'score': 89}
{'student': 'Correa_S3', 'subject': 'ELECTRONICS', 'score': 89}
{'student': 'Correa_S3', 'subject': 'PHYSICS', 'score': 90}
{'student': 'Correa_S3', 'subject': 'ENGLISH', 'score': 100}
{'student': 'Correa_S4', 'subject': 'PROGRAMMING', 'score': 91}
{'student': 'Correa_S4', 'subject': 'MATHEMATICS', 'score': 91}
{'student': 'Correa_S4', 'subject': 'ELECTRONICS', 'score': 92}
...
{'student': 'Correa_S5', 'subject': 'PHYSICS', 'score': 53, 'classification': 'NEEDS IMPROVEMENT'}
{'student': 'Correa_S5', 'subject': 'PROGRAMMING', 'score': 93, 'classification': 'EXCELLENT'}
Student performance data analysis completed successfully.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
Outline
Timeline
Lab4_DataStructures_Correa
activity-4*
Spaces: 4
LF
Cell 1 of 1

EXERCISE # 3 TRANSACTION HISTORY ANALYZER



```python
from collections import defaultdict

# STUDENT-SPECIFIC INPUTS
LAST_NAME = "Correa"
SEED_NUM = 6
FAVORITE_ARTIST = "Jennie"

# Convert artist's letters to numerical positions (A=1, B=2, ..., Z=26)
artist_values = [
    ord(char.upper()) - ord("A") + 1
    for char in FAVORITE_ARTIST if char.isalpha()
]

print("STUDENT INPUTS")
print(f"Last Name: {LAST_NAME}")
print(f"Seed Number: {SEED_NUM}")
print(f"Favorite Artist: {FAVORITE_ARTIST}")
print(f"Character Values: {artist_values}")


# 1. GENERATE TRANSACTION RECORDS
customers = [f"{LAST_NAME}_C{i}" for i in range(1, 6)]

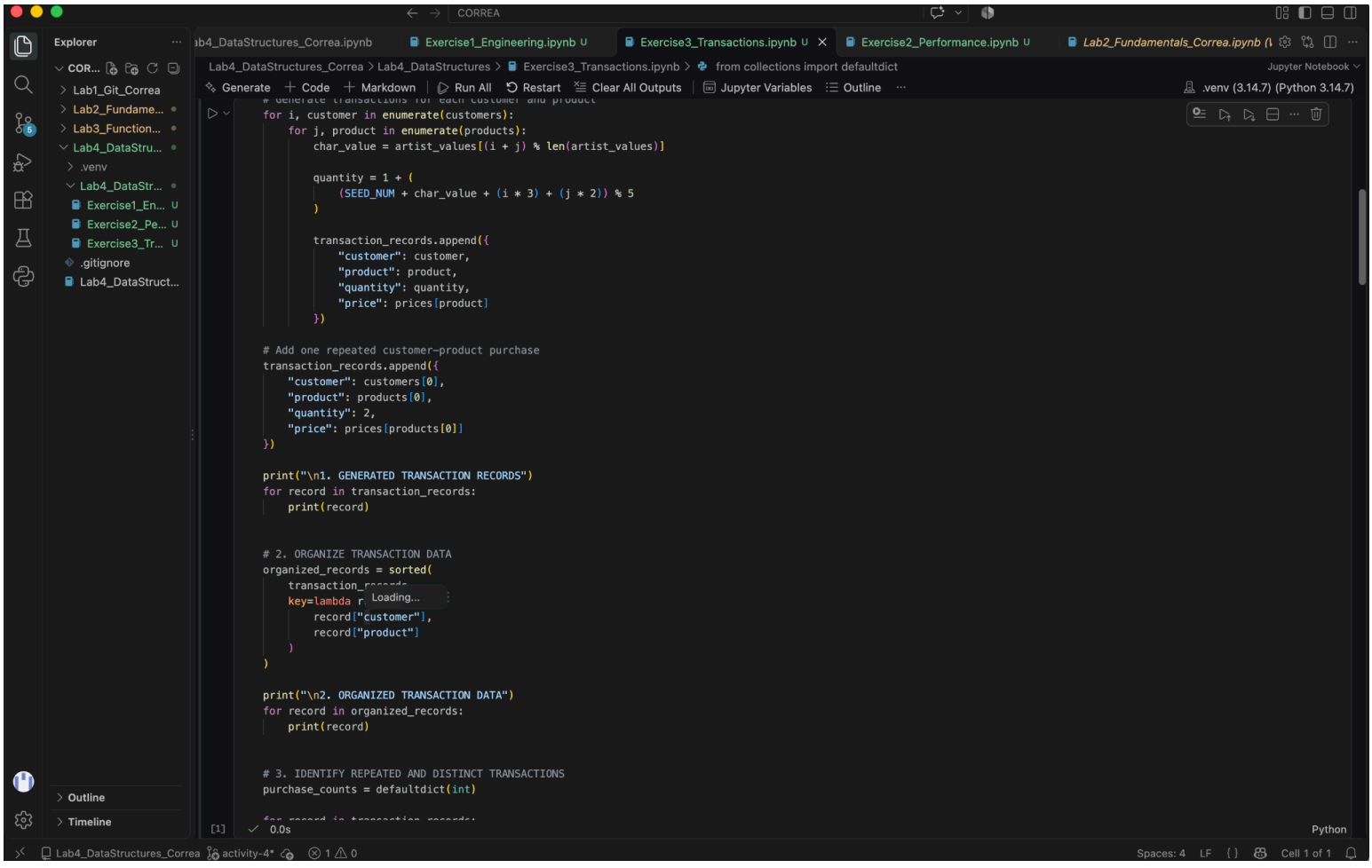
products = [
    "NOTEBOOK",
    "PEN",
    "USB",
    "HEADPHONES",
    "CALCULATOR"
]

# Fixed example prices for each product
prices = {
    "NOTEBOOK": 35.00,
    "PEN": 15.00,
    "USB": 250.00,
    "HEADPHONES": 350.00,
    "CALCULATOR": 180.00
}

transaction_records = []

# Generate transactions for each customer and product
for i, customer in enumerate(customers):
```

[1] 0.0s Python



```python
for i, customer in enumerate(customers):
    for j, product in enumerate(products):
        char_value = artist_values[(i + j) % len(artist_values)]

        quantity = 1 + (
            (SEED_NUM + char_value + (i * 3) + (j * 2)) % 5
        )

        transaction_records.append({
            "customer": customer,
            "product": product,
            "quantity": quantity,
            "price": prices[product]
        })

# Add one repeated customer-product purchase
transaction_records.append({
    "customer": customers[0],
    "product": products[0],
    "quantity": 2,
    "price": prices[products[0]]
})

print("\n1. GENERATED TRANSACTION RECORDS")
for record in transaction_records:
    print(record)

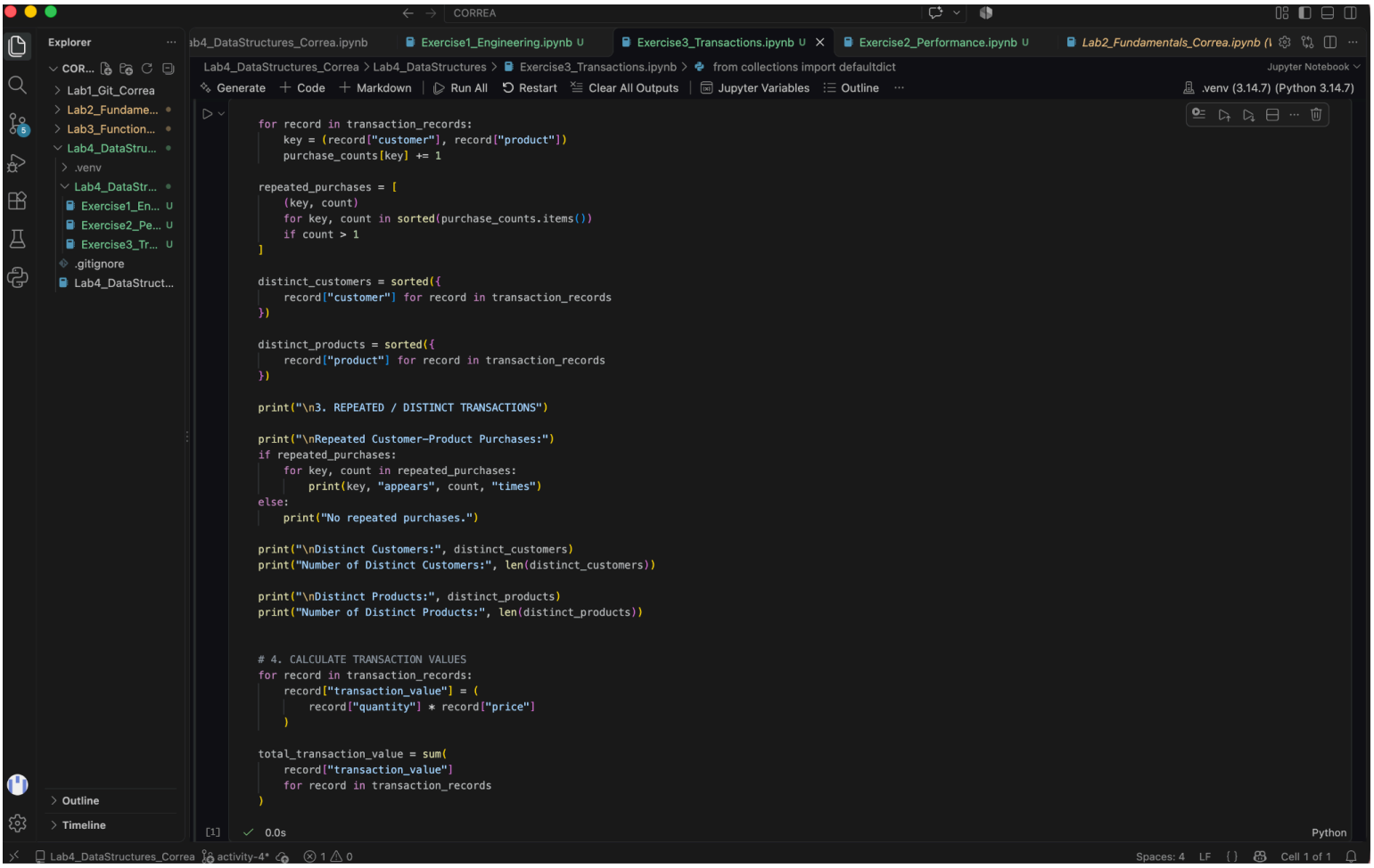

# 2. ORGANIZE TRANSACTION DATA
organized_records = sorted(
    transaction_records,
    key=lambda r: (
        record["customer"],
        record["product"]
    )
)

print("\n2. ORGANIZED TRANSACTION DATA")
for record in organized_records:
    print(record)


# 3. IDENTIFY REPEATED AND DISTINCT TRANSACTIONS
purchase_counts = defaultdict(int)
```

Loading...

[1] 0.0s Python



```
for record in transaction_records:
    key = (record["customer"], record["product"])
    purchase_counts[key] += 1

repeated_purchases = [
    (key, count)
    for key, count in sorted(purchase_counts.items())
    if count > 1
]

distinct_customers = sorted({
    record["customer"] for record in transaction_records
})

distinct_products = sorted({
    record["product"] for record in transaction_records
})

print("\n3. REPEATED / DISTINCT TRANSACTIONS")

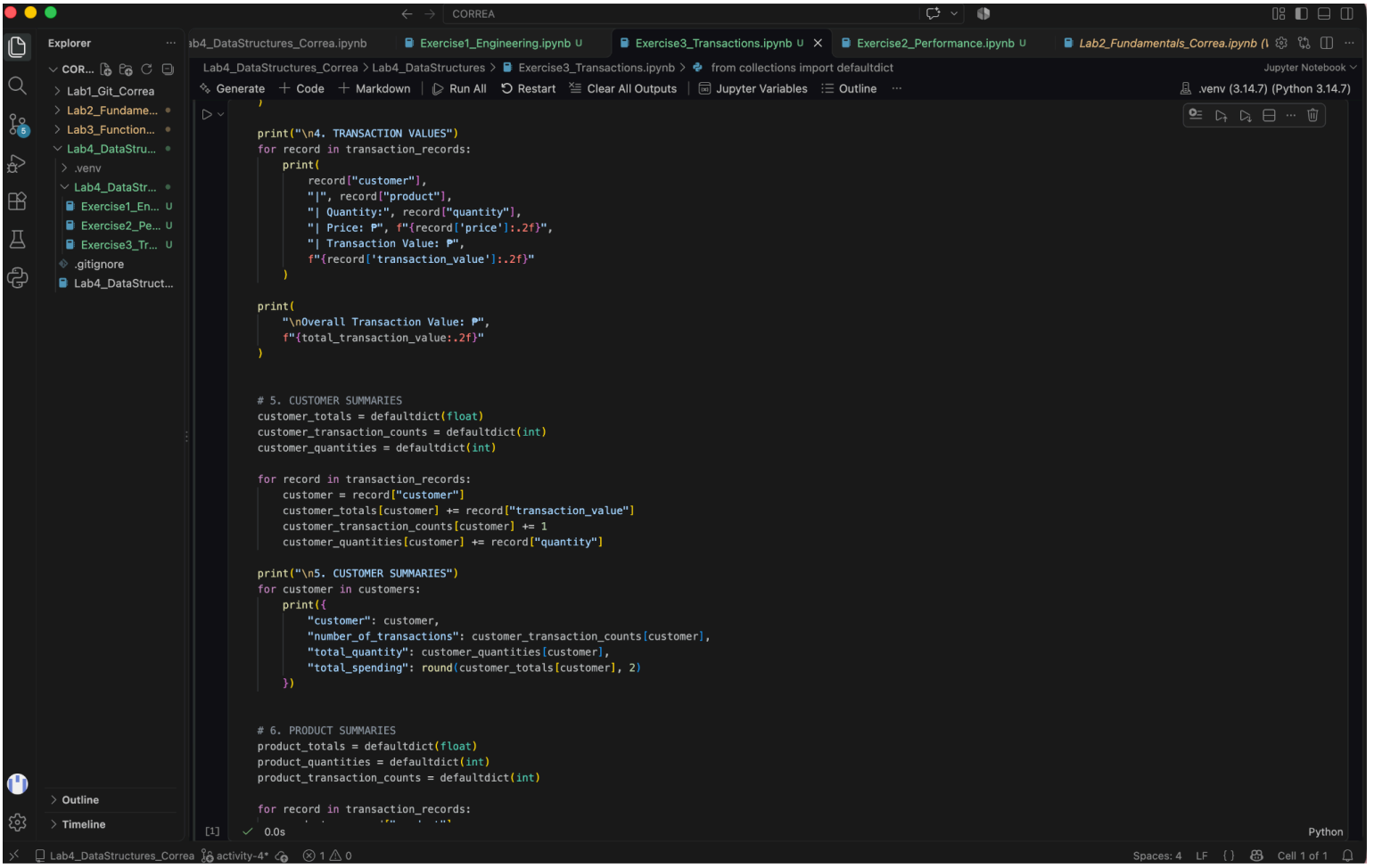
print("\nRepeated Customer–Product Purchases:")
if repeated_purchases:
    for key, count in repeated_purchases:
        print(key, "appears", count, "times")
else:
    print("No repeated purchases.")

print("\nDistinct Customers:", distinct_customers)
print("Number of Distinct Customers:", len(distinct_customers))

print("\nDistinct Products:", distinct_products)
print("Number of Distinct Products:", len(distinct_products))


# 4. CALCULATE TRANSACTION VALUES
for record in transaction_records:
    record["transaction_value"] = (
        record["quantity"] * record["price"]
    )

total_transaction_value = sum(
    record["transaction_value"]
    for record in transaction_records
)
```



```
)

print("\n4. TRANSACTION VALUES")
for record in transaction_records:
    print(
        record["customer"],
        "|", record["product"],
        "| Quantity:", record["quantity"],
        "| Price: ₱", f"{record['price']:.2f}",
        "| Transaction Value: ₱",
        f"{record['transaction_value']:.2f}"
    )

print(
    "\nOverall Transaction Value: ₱",
    f"{total_transaction_value:.2f}"
)


# 5. CUSTOMER SUMMARIES
customer_totals = defaultdict(float)
customer_transaction_counts = defaultdict(int)
customer_quantities = defaultdict(int)

for record in transaction_records:
    customer = record["customer"]
    customer_totals[customer] += record["transaction_value"]
    customer_transaction_counts[customer] += 1
    customer_quantities[customer] += record["quantity"]

print("\n5. CUSTOMER SUMMARIES")
for customer in customers:
    print({
        "customer": customer,
        "number_of_transactions": customer_transaction_counts[customer],
        "total_quantity": customer_quantities[customer],
        "total_spending": round(customer_totals[customer], 2)
    })


# 6. PRODUCT SUMMARIES
product_totals = defaultdict(float)
product_quantities = defaultdict(int)
product_transaction_counts = defaultdict(int)

for record in transaction_records:
```

```
# 6. PRODUCT SUMMARIES
product_totals = defaultdict(float)
product_quantities = defaultdict(int)
product_transaction_counts = defaultdict(int)

for record in transaction_records:
    product = record["product"]
    product_totals[product] += record["transaction_value"]
    product_quantities[product] += record["quantity"]
    product_transaction_counts[product] += 1

print("\n6. PRODUCT SUMMARIES")
for product in products:
    print({
        "product": product,
        "price": prices[product],
        "number_of_transactions": product_transaction_counts[product],
        "total_quantity_sold": product_quantities[product],
        "total_sales": round(product_totals[product], 2)
    })


# 7. ORDERED RESULTS
print("\n7. ORDERED RESULTS")

print("\nTransactions ordered by customer and product:")
for record in organized_records:
    print({
        **record,
        "transaction_value": round(
            record["quantity"] * record["price"], 2
        )
    })

print("\nTransactions ordered from highest to lowest value:")
ranked_records = sorted(
    transaction_records,
    key=lambda record: record["transaction_value"],
    reverse=True
)

for record in ranked_records:
    print(
        record["customer"],
        "|", record["product"],
```

```
# 8. OVERALL TRANSACTION SUMMARY
total_transactions = len(transaction_records)
total_units_sold = sum(
    record["quantity"] for record in transaction_records
)

average_transaction_value = (
    total_transaction_value / total_transactions
    if total_transactions > 0 else 0
)

highest_transaction = max(
    transaction_records,
    key=lambda record: record["transaction_value"]
)

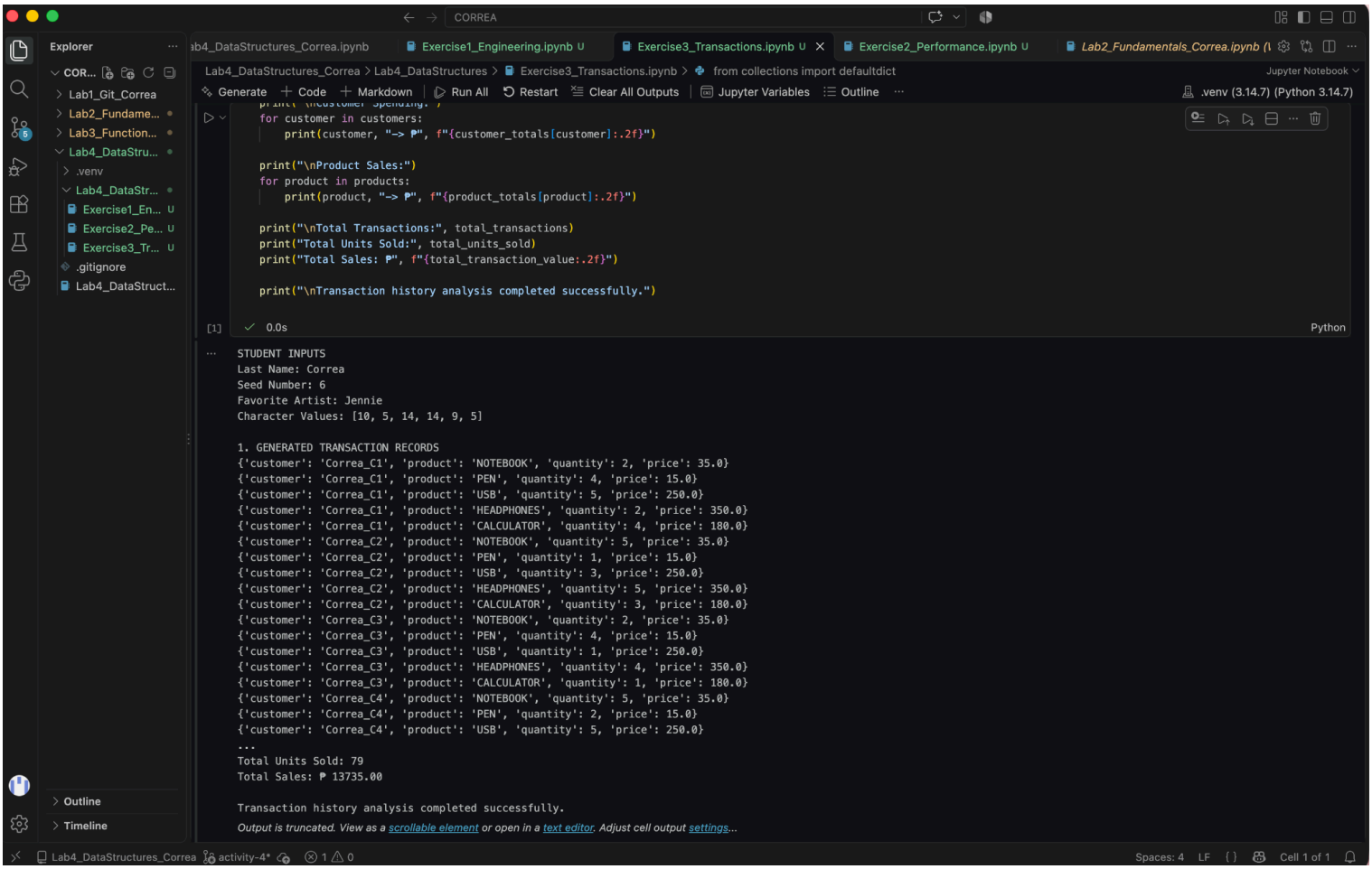
lowest_transaction = min(
    transaction_records,
    key=lambda record: record["transaction_value"]
)

print("\n8. OVERALL TRANSACTION SUMMARY")
print("Total Transaction Records:", total_transactions)
print("Distinct Customers:", len(distinct_customers))
print("Distinct Products:", len(distinct_products))
print("Repeated Customer-Product Pairs:", len(repeated_purchases))
print("Total Units Sold:", total_units_sold)
print("Overall Transaction Value: ₱", f"{total_transaction_value:.2f}")
print("Average Transaction Value: ₱", f"{average_transaction_value:.2f}")

print(
    "Highest Transaction:",
    highest_transaction["customer"],
    "|", highest_transaction["product"],
    "| ₱", f"{highest_transaction['transaction_value']:.2f}"
)

print(
    "Lowest Transaction:",
    lowest_transaction["customer"],
    "|", lowest_transaction["product"],
    "| ₱", f"{lowest_transaction['transaction_value']:.2f}"
)

print("\nFINAL CONSOLIDATED TRANSACTION REPORT")
```


CORREA
Explorer
Lab1_Git_Correa
Lab2_Fundame...
Lab3_Function...
Lab4_DataStru...
.venv
Lab4_DataStr...
Exercise1_En... U
Exercise2_Pe... U
Exercise3_Tr... U
.gitignore
Lab4_DataStruct...
Outline
Timeline
Exercise1_Engineering.ipynb U
Exercise3_Transactions.ipynb U
Exercise2_Performance.ipynb U
Lab2_Fundamentals_Correa.ipynb
Lab4_DataStructures_Correa > Lab4_DataStructures > Exercise3_Transactions.ipynb > from collections import defaultdict
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
.venv (3.14.7) (Python 3.14.7)
for customer in customers:
print(customer, "-> ₱", f"{customer_totals[customer]:.2f}")
print("\nProduct Sales:")
for product in products:
print(product, "-> ₱", f"{product_totals[product]:.2f}")
print("\nTotal Transactions:", total_transactions)
print("Total Units Sold:", total_units_sold)
print("Total Sales: ₱", f"{total_transaction_value:.2f}")
print("\nTransaction history analysis completed successfully.")
[1]
0.0s
Python
STUDENT INPUTS
Last Name: Correa
Seed Number: 6
Favorite Artist: Jennie
Character Values: [10, 5, 14, 14, 9, 5]
1. GENERATED TRANSACTION RECORDS
{'customer': 'Correa_C1', 'product': 'NOTEBOOK', 'quantity': 2, 'price': 35.0}
{'customer': 'Correa_C1', 'product': 'PEN', 'quantity': 4, 'price': 15.0}
{'customer': 'Correa_C1', 'product': 'USB', 'quantity': 5, 'price': 250.0}
{'customer': 'Correa_C1', 'product': 'HEADPHONES', 'quantity': 2, 'price': 350.0}
{'customer': 'Correa_C1', 'product': 'CALCULATOR', 'quantity': 4, 'price': 180.0}
{'customer': 'Correa_C2', 'product': 'NOTEBOOK', 'quantity': 5, 'price': 35.0}
{'customer': 'Correa_C2', 'product': 'PEN', 'quantity': 1, 'price': 15.0}
{'customer': 'Correa_C2', 'product': 'USB', 'quantity': 3, 'price': 250.0}
{'customer': 'Correa_C2', 'product': 'HEADPHONES', 'quantity': 5, 'price': 350.0}
{'customer': 'Correa_C2', 'product': 'CALCULATOR', 'quantity': 3, 'price': 180.0}
{'customer': 'Correa_C3', 'product': 'NOTEBOOK', 'quantity': 2, 'price': 35.0}
{'customer': 'Correa_C3', 'product': 'PEN', 'quantity': 4, 'price': 15.0}
{'customer': 'Correa_C3', 'product': 'USB', 'quantity': 1, 'price': 250.0}
{'customer': 'Correa_C3', 'product': 'HEADPHONES', 'quantity': 4, 'price': 350.0}
{'customer': 'Correa_C3', 'product': 'CALCULATOR', 'quantity': 1, 'price': 180.0}
{'customer': 'Correa_C4', 'product': 'NOTEBOOK', 'quantity': 5, 'price': 35.0}
{'customer': 'Correa_C4', 'product': 'PEN', 'quantity': 2, 'price': 15.0}
{'customer': 'Correa_C4', 'product': 'USB', 'quantity': 5, 'price': 250.0}
...
Total Units Sold: 79
Total Sales: ₱ 13735.00
Transaction history analysis completed successfully.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
Lab4_DataStructures_Correa
activity-4*
Spaces: 4
LF
Cell 1 of 1